

Heart Disease — Exploratory Data Analysis

Dataset: UCI Heart Disease (Cleveland processed)

Sections:

1. Loading dataset and basic inspection
2. Identifying missing values
3. Checking target / class balance / imbalance
4. Feature distributions (histograms / count plots)
5. Outlier inspection (boxplots by target)
6. Feature Correlation heatmap
7. Save processed CSV + EDA artifacts

```
In [1]: import sys
from pathlib import Path

# Make src/ importable when running from notebooks/
ROOT = Path.cwd().parent if Path.cwd().name == 'notebooks' else Path.cwd()
sys.path.insert(0, str(ROOT))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from src.data_loader import (
    load_raw, clean, save_processed, summary,
    NUMERIC_FEATURES, CATEGORICAL_FEATURES, TARGET,
)

sns.set_theme(style='whitegrid', context='notebook')
ARTIFACTS = ROOT / 'reports' / 'figures'
ARTIFACTS.mkdir(parents=True, exist_ok=True)
print('Artifacts ->', ARTIFACTS)
```

Artifacts -> C:\Users\vinogane\OneDrive - Cisco\Desktop\BITS-MTech\Semester-2\MLOps
\Assignment1\reports\figures

1. Loading Dataset and Basic Inspection

```
In [2]: raw = load_raw()
print('Data Input shape:', raw.shape)
raw.head()
```

Data Input shape: (303, 14)

Out[2]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0

In [3]:

```
df = clean(raw)
print('Cleaned Data shape:', df.shape)
print('\nSummary:', summary(df))
df.describe().T
```

Cleaned Data shape: (303, 14)

Summary: {'n_rows': 303, 'n_cols': 14, 'n_missing': 6, 'class_balance': {0: 0.5412541254125413, 1: 0.45874587458745875}}

Out[3]:

	count	mean	std	min	25%	50%	75%	max
age	303.0	54.438944	9.038662	29.0	48.0	56.0	61.0	77.0
sex	303.0	0.679868	0.467299	0.0	0.0	1.0	1.0	1.0
cp	303.0	3.158416	0.960126	1.0	3.0	3.0	4.0	4.0
trestbps	303.0	131.689769	17.599748	94.0	120.0	130.0	140.0	200.0
chol	303.0	246.693069	51.776918	126.0	211.0	241.0	275.0	564.0
fbs	303.0	0.148515	0.356198	0.0	0.0	0.0	0.0	1.0
restecg	303.0	0.990099	0.994971	0.0	0.0	1.0	2.0	2.0
thalach	303.0	149.607261	22.875003	71.0	133.5	153.0	166.0	202.0
exang	303.0	0.326733	0.469794	0.0	0.0	0.0	1.0	1.0
oldpeak	303.0	1.039604	1.161075	0.0	0.0	0.8	1.6	6.2
slope	303.0	1.600660	0.616226	1.0	1.0	2.0	2.0	3.0
ca	299.0	0.672241	0.937438	0.0	0.0	0.0	1.0	3.0
thal	301.0	4.734219	1.939706	3.0	3.0	3.0	7.0	7.0
target	303.0	0.458746	0.499120	0.0	0.0	0.0	1.0	1.0

2. Identifying Missing values

In [4]:

```
missing = raw.isna().sum().sort_values(ascending=False)
missing = missing[missing > 0]
```

```
print('Columns with missing values:')
print(missing)
print('\nTotal missing cells:', int(raw.isna().sum().sum()))
print('% rows with any missing:', round(raw.isna().any(axis=1).mean()*100, 2), '%')
```

Columns with missing values:

```
ca      4
thal    2
dtype: int64
```

Total missing cells: 6

% rows with any missing: 1.98 %

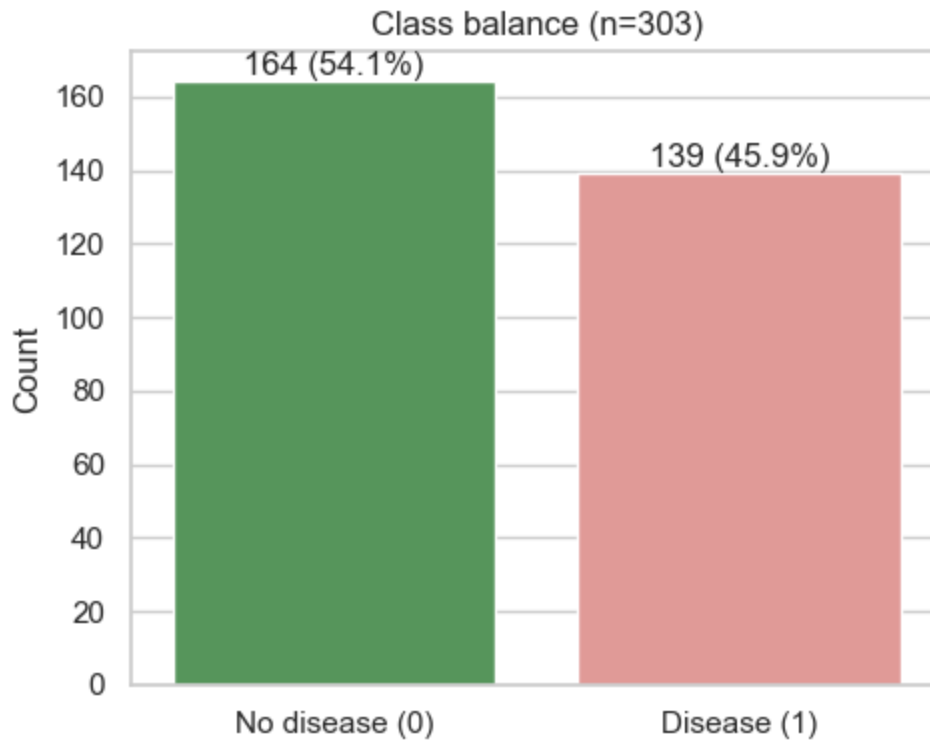
3. Checking Target / Class balance / Imbalance

```
In [5]: fig, ax = plt.subplots(figsize=(5, 4))
counts = df[TARGET].value_counts().sort_index()
sns.barplot(x=counts.index, y=counts.values, ax=ax,
            palette=["#4C9F52", "#EC8F8B"])
ax.set_xticklabels(['No disease (0)', 'Disease (1)'])
ax.set_ylabel('Count'); ax.set_xlabel('')
ax.set_title(f'Class balance (n={len(df)})')
for i, v in enumerate(counts.values):
    ax.text(i, v+2, f'{v} ({v/len(df)*100:.1f}%)', ha='center')
plt.tight_layout(); plt.savefig(ARTIFACTS/'class_balance.png', dpi=120); plt.show()
```

C:\Users\vinogane\AppData\Local\Temp\ipykernel_53132\1637453196.py:3: FutureWarning:

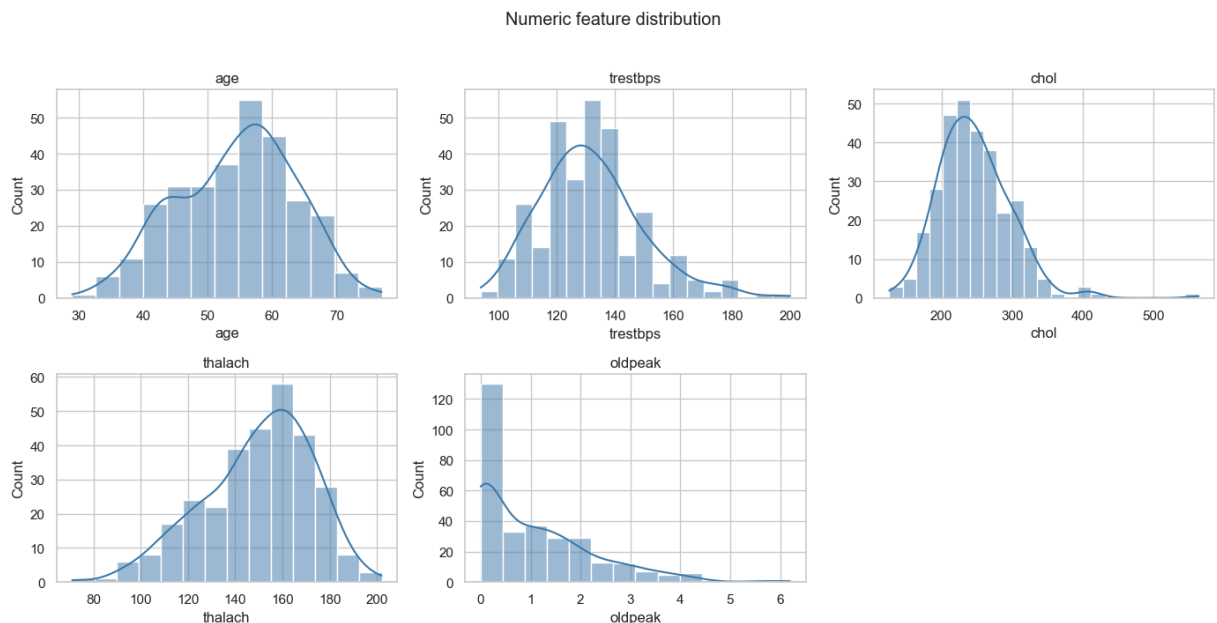
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=counts.index, y=counts.values, ax=ax,
C:\Users\vinogane\AppData\Local\Temp\ipykernel_53132\1637453196.py:5: UserWarning: s
et_ticklabels() should only be used with a fixed number of ticks, i.e. after set_tic
ks() or using a FixedLocator.
ax.set_xticklabels(['No disease (0)', 'Disease (1)'])
```

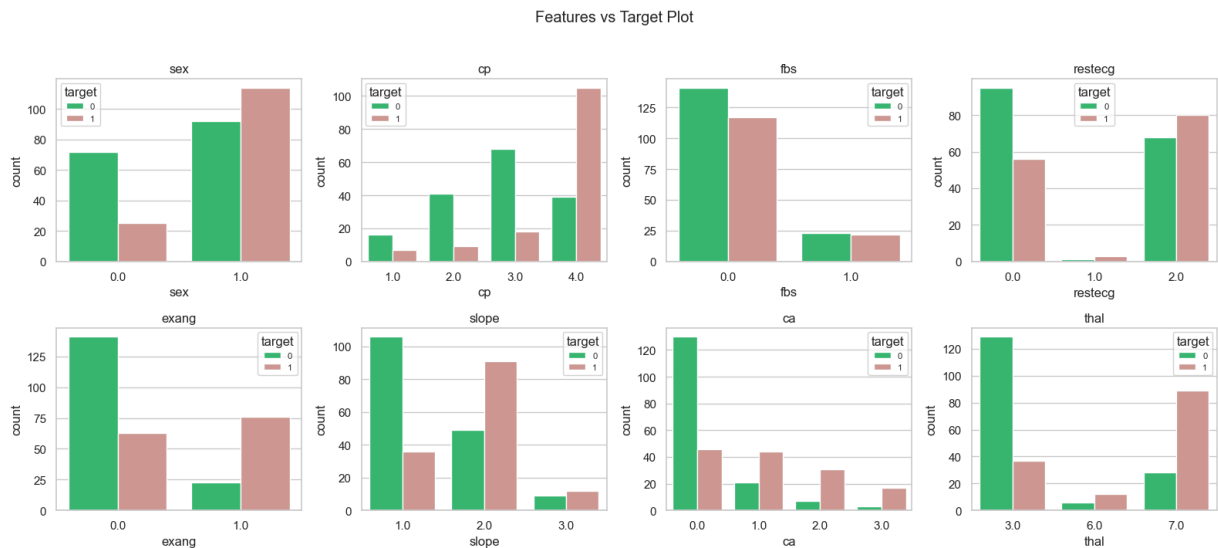


4. Feature distributions

```
In [6]: fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, col in zip(axes.flat, NUMERIC_FEATURES + ['']):
    if col == '':
        ax.axis('off'); continue
    sns.histplot(df[col].dropna(), kde=True, ax=ax, color='#3F7CAC')
    ax.set_title(col)
plt.suptitle('Numeric feature distribution', y=1.02)
plt.tight_layout(); plt.savefig(ARTIFACTS/'numeric_hist.png', dpi=120); plt.show()
```



```
In [7]: fig, axes = plt.subplots(2, 4, figsize=(16, 7))
        for ax, col in zip(axes.flat, CATEGORICAL_FEATURES):
            sns.countplot(data=df, x=col, hue=TARGET, ax=ax,
                           palette=["#23CD6D", "#D78D8A"])
            ax.set_title(col); ax.legend(title='target', fontsize=8)
        plt.suptitle('Features vs Target Plot', y=1.02)
        plt.tight_layout(); plt.savefig(ARTIFACTS/'categorical_vs_target.png', dpi=120); pl
```



5. Outlier inspection — boxplots of numeric features by target

Box plots show the median, IQR (25-75th percentile), whiskers ($1.5 \times \text{IQR}$) and individual points beyond the whiskers — i.e. **outliers**. Splitting by `target` also reveals whether the distribution shifts between the two classes.

```
In [8]: fig, axes = plt.subplots(2, 3, figsize=(14, 7))
        for ax, col in zip(axes.flat, NUMERIC_FEATURES + ['']):
            if col == '':
                ax.axis('off'); continue
            sns.boxplot(data=df, x=TARGET, y=col, ax=ax,
                         palette=["#4C9F70", "#D9534F"])
            ax.set_title(f'{col} by target')
        plt.tight_layout(); plt.savefig(ARTIFACTS/'numeric_vs_target.png', dpi=120); plt.sh
```

```
C:\Users\vinogane\AppData\Local\Temp\ipykernel_53132\136777473.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x=TARGET, y=col, ax=ax,
```

```
C:\Users\vinogane\AppData\Local\Temp\ipykernel_53132\136777473.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x=TARGET, y=col, ax=ax,
```

```
C:\Users\vinogane\AppData\Local\Temp\ipykernel_53132\136777473.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x=TARGET, y=col, ax=ax,
```

```
C:\Users\vinogane\AppData\Local\Temp\ipykernel_53132\136777473.py:5: FutureWarning:
```

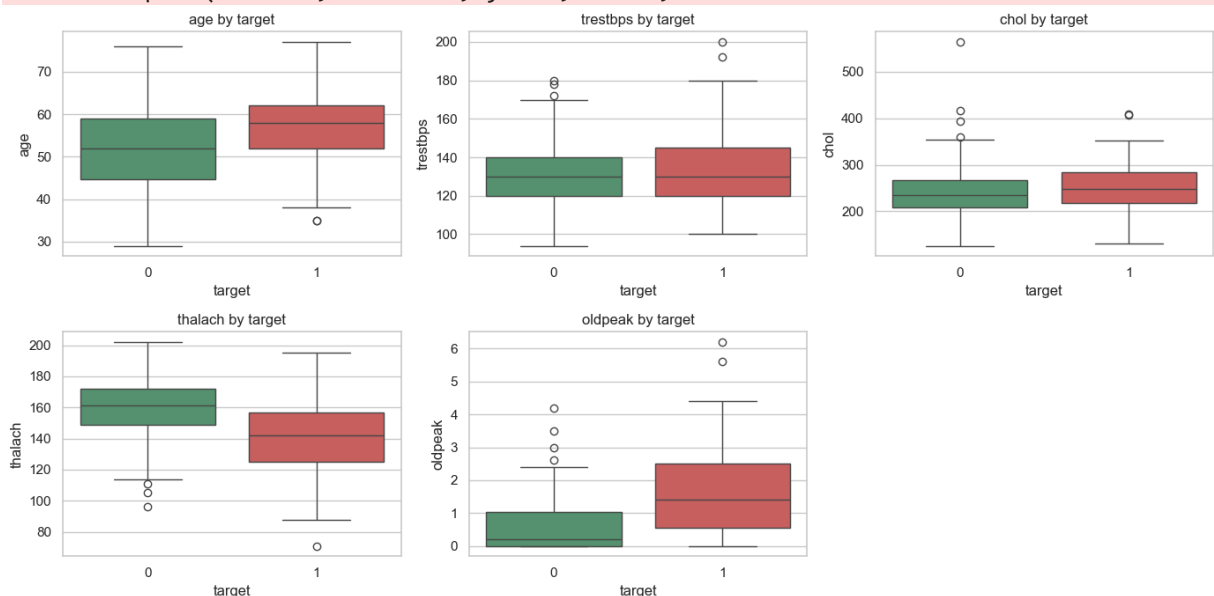
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x=TARGET, y=col, ax=ax,
```

```
C:\Users\vinogane\AppData\Local\Temp\ipykernel_53132\136777473.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data=df, x=TARGET, y=col, ax=ax,
```



6. Feature Correlation heatmap

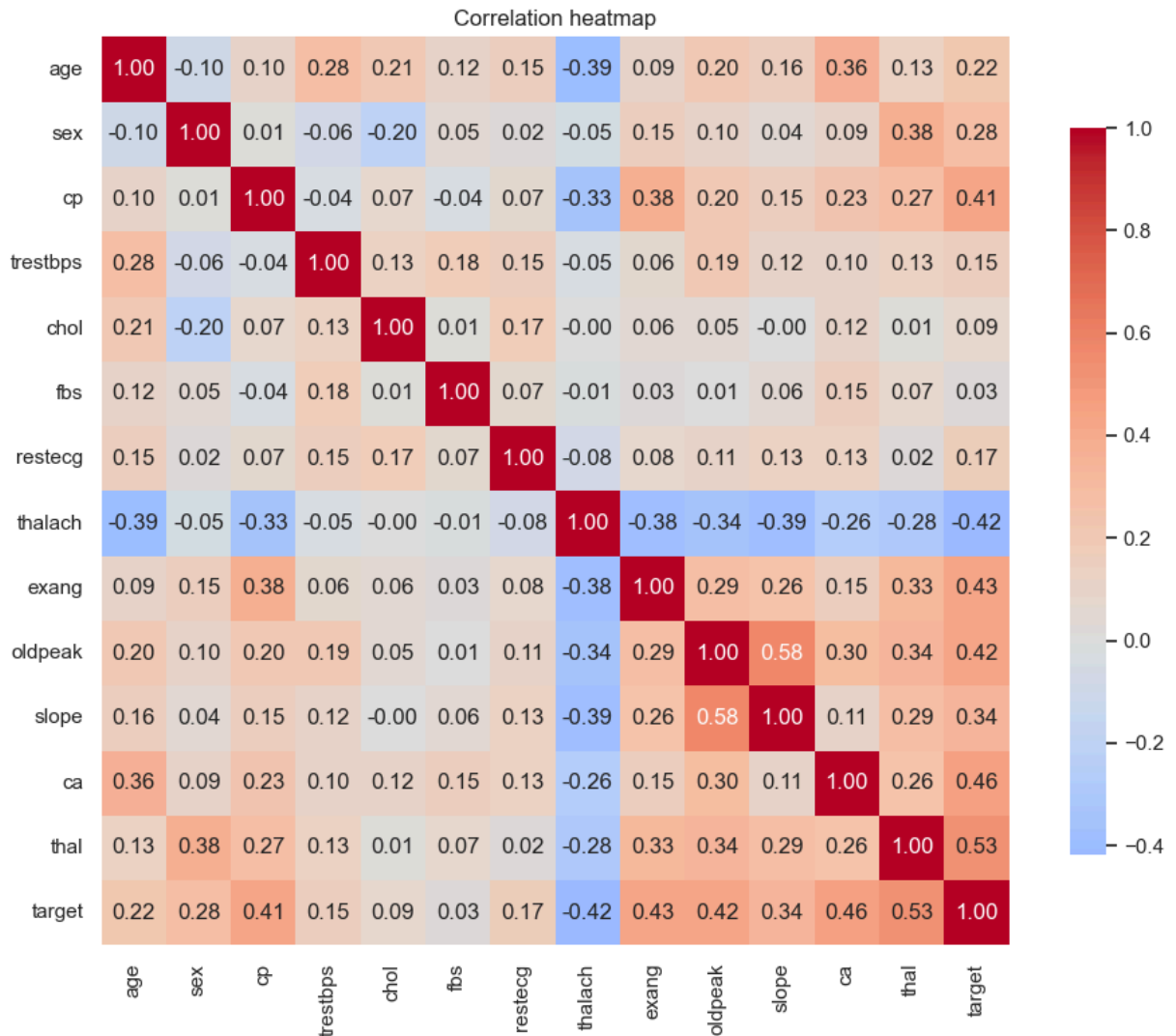
```
In [9]: fig, ax = plt.subplots(figsize=(10, 8))
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
```

```

        center=0, ax=ax, square=True, cbar_kws={'shrink': 0.8})
ax.set_title('Correlation heatmap')
plt.tight_layout(); plt.savefig(ARTIFACTS/'correlation_heatmap.png', dpi=120); plt.

print('\nTop correlations with target:')
print(corr[TARGET].drop(TARGET).abs().sort_values(ascending=False).head(8))

```



Top correlations with target:

```

thal      0.525689
ca        0.460442
exang     0.431894
oldpeak   0.424510
thalach   0.417167
cp        0.414446
slope     0.339213
sex       0.276816

```

Name: target, dtype: float64

7. Save processed CSV

```

In [10]: out = save_processed(df)
print('Processed dataset ->', out)

```

```
print('Figures ->', ARTIFACTS)
```

Processed dataset -> C:\Users\vinogane\OneDrive - Cisco\Desktop\BITS-MTech\Semester-2\MLOps\Assignment1\data\processed\heart_cleveland.csv

Figures -> C:\Users\vinogane\OneDrive - Cisco\Desktop\BITS-MTech\Semester-2\MLOps\Assignment1\reports\figures

Key findings

- **Size:** 303 rows × 14 columns; nearly balanced classes ($\approx 54\%$ / 46%).
- **Missing values:** Only 6 missing cells across `ca` and `thal` — handled by median/mode imputation in the preprocessing pipeline.
- **Strongest associations** with the target are typically `cp` (chest pain type), `thalach` (max heart rate), `oldpeak`, `exang`, `ca`, and `thal`.
- **Scales differ widely** (`age` ~30-80 vs `oldpeak` ~0-6) → standard scaling required for distance-based / linear models.
- **Categorical variables** are integer-encoded but nominal (`cp`, `restecg`, `slope`, `thal`) → one-hot encode in the pipeline.